

# Clase I: Formalidades, Intro a IA y ML

**Aprendizaje automático:** resolución de problemas con programación tradicional. Un algoritmo que **aprende solo**.

→ ¿Cuál es una silla?

- 1) Se puede llamar a un carpintero y que nos diga. Se genera un sistema experto pero que no es escalable.
- 2) Juntar objetos que sí sean sillas y otros objetos que no. Un algoritmo **reconocerá patrones**. Esto es un aprendizaje automático.

Tipos de aprendizaje automático:

- 1) Supervisado: para cada observación hay una **respuesta**.
  - a) Silla de madera → “Es una silla”
  - b) Planta → “No es una silla”
- 2) No supervisado: no hay respuestas.
- 3) Por refuerzos: maximización de beneficios interactuando con un ambiente.

En el aprendizaje supervisado se quiere **predecir una variable** concreta.

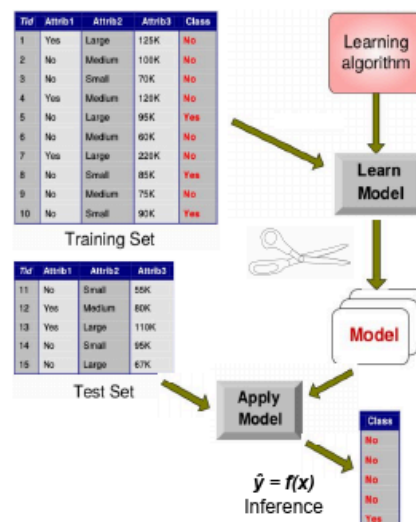
Hay dos tipos de variables para predecir:

- 1) **Continua** → **Regresión**
- 2) **Categorica** → **Clasificación** (binaria, multiclase)

Hay una **función  $f$  que no conocemos** que se aplica sobre el training set más un ruido que **genera la respuesta**. Queremos **estimar  $f$**  de la mejor manera posible y poder predecir de la mejor manera.

## 1) Aprendizaje / Entrenamiento

## 2) Inferencia / Predicción



Problema de regresión:

→ Predecir un valor numérico  $\hat{y}_i$  lo más cercano posible al valor real  $y_i$ .

Problema de clasificación:

→ Estimar las probabilidades  $\hat{P}(y = k | X_i)$  para cada clase  $k$ .

→ Si la clase real de  $i$  es  $k$ , queremos que  $\hat{P}(y = k | X_i) \approx 1$  y que las otras probabilidades estén cerca de 0. La probabilidad estima la clasificación.

| Animal           | ¿Da a luz? | ¿Vuela? | ¿Vive en el agua? | ¿Tiene piernas? | ¿Mamífero?  |
|------------------|------------|---------|-------------------|-----------------|-------------|
| humano           | sí         | no      | no                | sí              | mamífero    |
| pitón            | no         | no      | no                | no              | no-mamífero |
| salmon           | no         | no      | sí                | no              | no-mamífero |
| ballena          | sí         | no      | sí                | no              | mamífero    |
| rana             | no         | no      | a veces           | sí              | no-mamífero |
| dragón de komodo | no         | no      | no                | sí              | no-mamífero |
| murciélago       | sí         | sí      | no                | sí              | mamífero    |
| paloma           | no         | sí      | no                | sí              | no-mamífero |
| gato             | sí         | no      | no                | sí              | mamífero    |
| tiburón leopardo | sí         | no      | sí                | no              | no-mamífero |
| tortuga          | no         | no      | a veces           | sí              | no-mamífero |
| pingüino         | no         | no      | a veces           | sí              | no-mamífero |
| puercoespín      | sí         | no      | no                | sí              | mamífero    |
| anguila          | no         | no      | sí                | no              | no-mamífero |
| salamandra       | no         | no      | a veces           | sí              | no-mamífero |
| monstruo de gila | no         | no      | no                | sí              | no-mamífero |
| ornitorrinco     | no         | no      | no                | sí              | mamífero    |
| búho             | no         | sí      | no                | sí              | no-mamífero |
| delfín           | sí         | no      | sí                | no              | mamífero    |
| águila           | no         | sí      | no                | sí              | no-mamífero |

| ¿Da a luz? | ¿Vuela? | ¿Vive en el agua? | ¿Tiene piernas? | ¿Mamífero? |
|------------|---------|-------------------|-----------------|------------|
| sí         | no      | sí                | no              | ¿?         |

El cuadro de arriba es el dataset/**training set X** con sus respuestas asociadas.

→ Las columnas son las **variables**. Cada variable tiene un valor.

→ Las filas, las **observaciones**.

→ La columna “¿Mamífero?” es la **respuesta y**.

El cuadro de abajo es el test set. Queremos que nuestro modelo de aprendizaje automático **acierte** la respuesta de si es mamífero o no.

## Clase II: Árboles de Decisión

→ El training test y el test set son mutuamente excluyentes. El modelo debería ponerse a prueba con datos que no haya visto antes para que no haya sesgos.

→ Filas: observaciones. Columnas: variables.

→ Los posibles valores de las variables es el **espacio de predictores**.

Una observación se marca como un punto en ese espacio de predictores, ya que es una combinación de las mismas.

Primer modelo de aprendizaje supervisado: **árboles de decisión**.

→ Para clasificación binaria y multiclase.

→ También para regresión.

1) Para regresión: predecir una variable continua.

a) Medida de calidad de predicción: **suma de errores al cuadrado** (SSE).

$$\rightarrow SSE = \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$$

→ ¿Qué tan distinta es la predicción del valor real?

b) Se busca minimizar el SSE **sobre el training set**.

c) Un árbol particiona el espacio de predictores en **regiones**.

→ Los **nodos** internos dividen el espacio de predictores.

→ Las **hojas** dice en qué región cae una observación.

→ Las **regiones** son rectángulos jerárquicos paralelos a los ejes. Sesgo inductivo.

→ Por región hay un error de predicción llamado SSE en entrenamiento.

d) Se busca un **conjunto de regiones que minimicen** el SSE por cada región.

- e) Las regiones se consiguen de manera recursiva. ¿Dónde hago una partición? Donde se consiga mayor **ganancia**. Seguir hasta llegar a algún criterio de parada.
  - f) El árbol detecta interacciones entre predictores y no linealidades.
- 2) Para clasificación:
- a) Predicen **probabilidades de clase**.  
→  $\hat{P}(y = k | x)$  es la proporción de observaciones de la clase  $k$  en la región.
  - b) No se usa SSE. Se usan **medidas de impureza**.  
→ Gini, entropía y error de clasificación.
  - c) **Costo total del árbol**: suma de impureza de las hojas, ponderadas por su tamaño.
  - d) Pueden trabajar con **variables categóricas**, ya sea con splits múltiples (chico/mediano/grande) o cortes binarios (si/no).
  - e) **Flexibilidad**: capacidad de adaptación al training set.  
→ Árbol profundo  $\Rightarrow$  más flexible.  
→ Pero sobreajusta  $\Rightarrow$  capaz predice mal en el test set.  
→ Ajustar perfecto en training set **NO IMPLICA** que ajuste perfecto en test set.
  - f) Control de complejidad del árbol. Antes de entrenarlo definir:  
→ **max\_depth**: profundidad máxima.  
→ **min\_samples\_split**: mínimo de observaciones en un nodo de decisión para realizar un split.  
→ **min\_samples\_leaf**: mínimo de observaciones que deben tener las hojas.  
→ **min\_impurity\_decrease**: ganancia mínima para aceptar un split.  
→ Técnica pre-pruning: no permitir splits que no reduzcan lo suficiente el error.  
→ Técnica post-pruning: podar hojas luego de crecer el árbol completo, usando parámetro  $\alpha$ .

### Missings:

- 1) En entrenamiento: se ignoran observaciones con NA al calcular la ganancia de un split  $\Rightarrow$  el split se elige solo con datos completos.
- 2) En predicción: si el valor de split está faltante, se usan **surrogate splits** (particiones sustitutas) que intentan imitar el split original.  
→ Se rankean según qué tan bien replican la división.  
→ Si no hay surrogate útil, se aplica **regla de la mayoría** (mandar la observación al lado con más datos).

| Ventajas                                                                  | Desventajas                                                                                            |
|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Intuitivos y fáciles de explicar.                                         | En predicción, suelen ser superados por modelos más avanzados (ej., Random Forests, XGBoost).          |
| Reflejan bien la lógica de "decisión humana".                             | Poco robustos: cambios pequeños en los datos $\rightarrow$ cambios grandes en la estructura del árbol. |
| Visualizables gráficamente $\rightarrow$ interpretables para no expertos. |                                                                                                        |
| Manejo bueno de missings.                                                 |                                                                                                        |
| Soportan naturalmente variables categóricas.                              |                                                                                                        |

## Clase III: Overfitting y Validación de Modelos

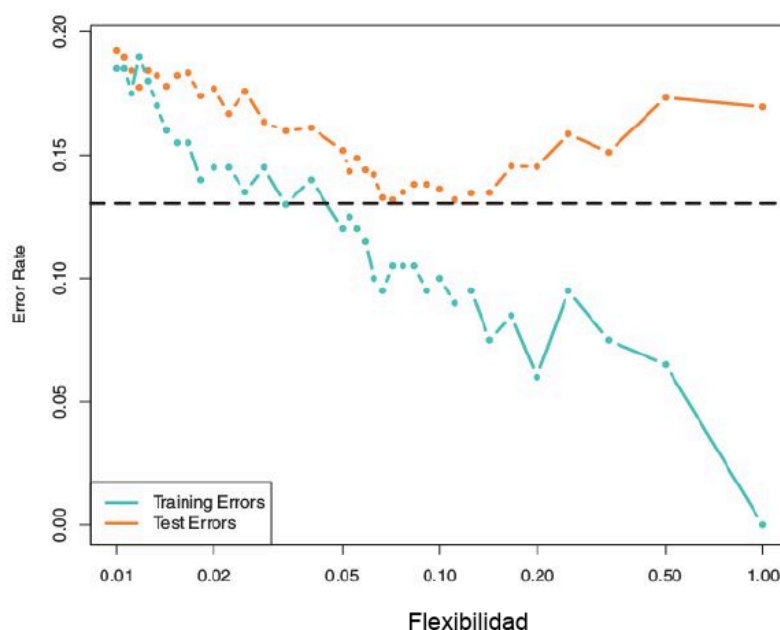
→ El modelo que vamos a usar DEPENDE de:

- 1) Qué training set se usó.
- 2) Qué algoritmo de aprendizaje se usó.

Podemos tener mayor o menor flexibilidad en el modelo sin importar con qué algoritmos lo entrenamos.

**Underfitting** → modelo rígido que no captura patrones del training set. Mal performance en training y testing.

**Overfitting** → modelo demasiado flexible. Ajusta demasiado las particularidades. Buena performance en training y mala en testing.



+ Flexibilidad  
- Error en training  
+ Error en test

Hay que **elegir bien** qué algoritmos, hiperparámetros, datos, pre-procesamiento usar antes de modelar porque influye bastante en el modelo resultante.

Queremos que funcione **bien en training** (que estime bien  $f$ ) y **también en testing**.

**Model validation** → estimar performance de un modelo sobre varios modelos "candidatos" para quedarnos

con el mejor.

- 1) Setup:
  - a) Proceso generador de datos desconocido.
  - b) Tenemos observaciones en el training set generadas por este proceso.  $X_{tr}$  e  $y_{tr}$ .
  - c) Relación  $y_{tr} = f(X_{tr}) + \epsilon$ . No conocemos  $f$  ni  $\epsilon$  (ruido).
- 2) Objetivo:
  - a) El proceso genera nuevas observaciones → **test set**  $(X_{ts}, y_{ts})$ .
  - b) Cada observación tiene sus variables. Conocemos  $X_{ts}$  no más.
  - c) Queremos estimar bien  $y_{ts}$  (la respuesta).
- 3) Approach:
  - a) Entrenar un modelo basándonos en  $X_{tr}$  e  $y_{tr}$ .
  - b) Se obtiene una estimación de  $f$  llamada  $\hat{f}$ . La vamos a usar.

El azar importa → El  $X_{tr}$  como el  $X_{ts}$  son datos random del proceso. Si hubiéramos tenido otros, las predicciones y la performance variarían.

**Queremos conocer el error de test**, pero como no tenemos acceso infinito a  $X_{ts}$ , se **estima** ese error con distintos métodos de validación.

Métodos de validación:

- 1) **Validation/Holdout set**
  - a) Separar al azar parte de los datos para validar.
  - b) Pros: simple.
  - c) Contras: depende de la partición, puede ser pesimista si hay pocos datos.
- 2) **Leave-One-Out Cross Validation (LOOCV)**
  - a) Validar dejando una observación afuera cada vez.
  - b) Pros: usa casi todos los datos para entrenar.
  - c) Contras: computacionalmente caro.
- 3) **k-fold Cross-Validation**
  - a) Dividir en  $k$  partes, entrenar en  $k - 1$  y validar en la restante.
  - b) Valores típicos:  $k = 5$  o  $10$ .
  - c) Puede repetirse varias veces para reducir variabilidad.

**Model selection** → elegir el mejor modelo comparando performance (train/valid/test).

**Model assessment** → estimar error de generalización del modelo elegido.

---

## Clase IV: Métricas de Performance

Vamos a usar métricas para ver qué tan bueno es nuestro modelo. Qué tan bien se logra el objetivo apuntado, ya sea en regresión o en clasificación.

Las métricas cuantifican la performance del modelo en distintos datasets y guían el entrenamiento en ciertos modelos.

Las métricas usadas para guiar el entrenamiento y para testear no necesariamente deben ser las mismas.

**Métricas para regresión:**

- 1) Suma de Errores al Cuadrado (**SSE**).
- 2) Mean Squared Error (**MSE**).
  - a) Para que el error no dependa de la cantidad de observaciones.
$$\rightarrow MSE = \frac{1}{n} SSE$$
- 3) Root Mean Squared Error (**RMSE**).
  - a) Para expresar el valor en la unidad original.
$$\rightarrow RMSE = \sqrt{\frac{1}{n} SSE}$$
- 4) Mean Absolute Error (**MAE**).
  - a) Robustez ante valores extremos.

$$\rightarrow MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{f}(x_i)|$$

5) Root Mean Squared Logarithmic Error (**RMSLE**)

- a) Captar sutilezas relevantes del dataset.

$$\rightarrow RMSLE = \sqrt{\frac{1}{n} \sum_{i=1}^n \left[ \ln(y_i + 1) - \ln(\hat{f}(x_i + 1)) \right]^2}$$

**Métricas para clasificación:**

**Threshold** → umbral para decidir si una observación es verdadera o falsa. Da lugar a la matriz de confusión.

|              |   | Predicted class |    | Total instances |
|--------------|---|-----------------|----|-----------------|
|              |   | +               | -  |                 |
| Actual class | + | TP              | FN | P               |
|              | - | FP              | TN | N               |

1) **Accuracy:**

- a) Mide la cantidad de aciertos.  
b) Depende del threshold elegido.

$$\rightarrow accuracy = (TP + TN) / (P + N)$$

2) **F1-score:**

- a) Ponderación de precision y recall.  
b) Depende del threshold elegido.  
c) Precision:

→ De los que dije que son verdaderos, ¿cuántos realmente son verdaderos?

$$\rightarrow Precision = TP / (TP + FP)$$

- d) Recall:

→ De los que realmente son verdaderos, ¿cuántos dije que son verdaderos?

$$\rightarrow Recall = TP / (TP + FN)$$

- e) Hay un tradeoff entre el precision y recall si se modifica el threshold.

$$\rightarrow F_1 score = \frac{2 \times precision \times recall}{precision + recall}$$

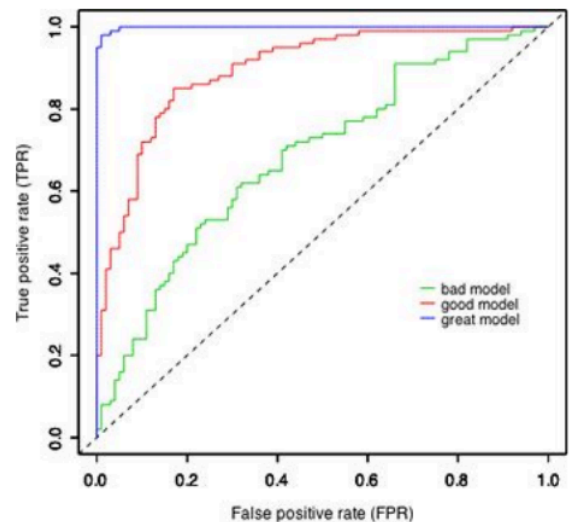
3) **Cross-entropy:**

- a) Para guiar construcción y evaluación de modelos.  
b) Para una observación  $i$ , se va a tomar un valor alto si se predice una probabilidad baja a la clase correcta.  
c) Hay una fórmula para clasificación binaria y otra para clasificación multiclase.

4) **ROC-AUC:**

- a) No depende del threshold.  
b) Relación entre **True Positive Rate** (TP/P) y **False Positive Rate** (FP/N).  
c) Se mide el área bajo la curva  $A$ .  
i)  $A < 0,5$  → no se puede: es peor que el azar  
ii)  $A = 0,5$  → azar: muy malo  
iii)  $0,5 < A < 1$  → modelo “bueno” o “malo”  
iv)  $A = 1$  → modelo perfecto

5) **Curva PR-AUC:**



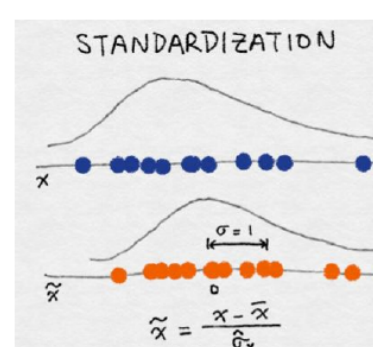
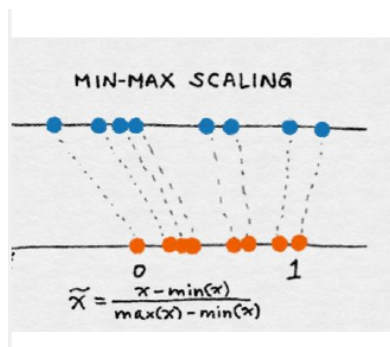
- a) Gráfico de valores de precision y recall a medida que se cambia el threshold.
- b) La cantidad de área bajo la curva también importa.

## Clase V: Ingeniería de Atributos

**Garbage in - Garbage out** → lo que aprenda un modelo depende de las variables. Se recomienda siempre evaluar con un esquema robusto (holdout, k-fold, ...) si las nuevas variables realmente mejoran la performance.

### Técnicas con variables numéricas:

- 1) **Log-transformation:**
  - a) Estandarización de escala.
  - b) Para variables con distribución asimétrica.
- 2) **Escalamiento y normalización:**
  - a) Para modelos sensibles a la varianza.
  - b) No modifica la forma de la distribución.
  - c) Modifica los valores de los ejes.



- 3) **Binarización:**
  - a) Convertir en variable 0/1 según un threshold.
- 4) **Binning:**
  - a) Agrupación de observaciones en bins (intervalos).
  - b) Genera variables categóricas ordinales.

### Técnicas con missings:

- 1) Tipos:
  - a) **Missing Completely at Random (MCAR)**
    - i) P(missing) no depende de ninguna variable.  
→ Alguien se olvidó de indicar su trabajo.
  - b) **Missing at Random (MAR)**
    - i) P(missing) depende de valores de otras variables.  
→ Las mujeres no suelen decir cuántos años tienen.
  - c) **Missing not at Random (MNAR)**
    - i) P(missing) depende de los valores de la misma variable.  
→ Los que tienen mucha plata no declaran sus ingresos.



| ID | Age | Gender | Annual income | Marital status | Number of children | Job      | Buy? |
|----|-----|--------|---------------|----------------|--------------------|----------|------|
| 1  |     | A      | 150,000       |                | 1                  | Engineer | No   |
| 2  | 27  | B      | 50,000        |                |                    | Teacher  | No   |
| 3  |     | A      | 100,000       | Married        | 2                  |          | Yes  |
| 4  | 40  | B      |               |                | 2                  | Engineer | Yes  |
| 5  | 35  | B      |               | Single         | 0                  | Doctor   | Yes  |
| 6  |     | A      | 50,000        |                | 0                  | Teacher  | No   |
| 7  | 33  | B      | 60,000        | Single         |                    | Teacher  | No   |
| 8  | 20  | B      | 10,000        |                |                    | Student  | No   |

- 2) Estrategias:
- Eliminar filas y columnas con missings (cuidado).
  - NA pasa a ser una categoría.
  - Llenar los missings a mano con media, mediana, moda, ...
  - Agregar una columna indicadora de missing si se imputan valores.

### Técnicas con variables categóricas:

- 1) **One-Hot Encoding (OHE):**
  - a) Transformar una variable categórica en **varias variables binarias** (0/1), una por cada categoría posible.
  - b) Permite que los algoritmos que sólo trabajan con números usen variables categóricas sin introducir un orden artificial.
  - c) Desventaja: **Curse of Dimensionality** → si hay demasiadas categorías, **explota el número de columnas**. Puede introducir mucha esparsidad (matrices con muchos ceros).
  - d) Si hay muchas categorías conviene usar matrices densas (que guarde solo los 1s).
- 2) **Diferenciación entre variables categóricas nominales y ordinales:**
  - a) Las nominales no tienen orden → (país) Italia, España, Mónaco.  
**Conviene usar OHE** con esas porque no hay jerarquías.
  - b) Las ordinales sí tienen orden → (educación) primaria < secundaria < universitaria. OHE las trata como independientes → **no es buena idea**.
    - i) Al modelo le costaría entender que “universitaria” está más cerca de “secundaria” que de “primaria”.
    - ii) **Conviene mapear categorías a números** → primaria = 1, secundaria = 2, universitaria = 3. Se entiende la jerarquía.
  - c) Nominales → OHE.  
Ordinales → mapeo numérico.
- 3) **Estrategias derivadas:**
  - a) Queremos generar nuevas variables numéricas a partir de categorías.
  - b) Tipos:
    - i) **Conteos:** ¿cuántas veces aparece una categoría en el dataset?
    - ii) **Target encoding:** reemplazar una categoría por la media (o proporción) del valor de la variable objetivo y en esa categoría.



→ si Género = F y en training el 70% de las mujeres compran ( $y=1$ ), se crea una variable "prob\_compra" = 0.7.

- 4) **Otras estrategias (para variables categóricas o numéricas):**
- a) **Interacciones (feature crossing):** productos o combinaciones polinómicas de variables.
    - i) Puede mejorar modelos lineales.
    - ii) Riesgo de overfitting.
  - b) **Variables temporales:** extraer año, mes, día, hora, diferencias entre timestamps, antigüedad, etc.
  - c) **Panel data:** medias móviles, máximos/mínimos móviles, tendencias.

---

## Clase VI: Selección de Atributos y Data Leakage

**Objetivo principal** → reducir la cantidad de variables

¿Por qué reducir la cantidad de variables?

- 1) **Mejorar performance** del modelo
- 2) Conseguir un modelo **más interpretable**

¿Qué variables sacamos?

- 1) Variables **redundantes:** tienen la misma información que otras variables.
- 2) Variables **irrelevantes:** no aportan valor (el promedio de los dígitos del DNI).
- 3) Variables **pesadas** para trabajar (ej. un dataset grande).

### Métodos de selección:

- 1) **Embedded methods:**
  - a) La selección se hace dentro del mismo entrenamiento.
    - i) **Árboles de decisión:** selección de atributos más relevantes al hacer splits.
    - ii) **Regresión Ridge:** penaliza coeficientes grandes acercándolos a cero (nunca llegan a ser cero). Se usan todas las variables siempre. Penalización tibia.

$$(1) \text{SSE}_{\text{Ridge}} = \sum_{i=1}^n (y_i - \hat{f}(x_i))^2 + \lambda \sum_{j=1}^p \beta_j^2$$

(2)  $\lambda = 0$  → es un SSE común

(3)  $\lambda = \infty$  → los coeficientes tienden a cero. NO EXACTAMENTE.

- iii) **Regresión Lasso:** penaliza valores absolutos y puede llevar coeficientes a exactamente cero. Puede no usar todas las variables. Penalización en serio.

$$(1) \text{SSE}_{\text{Lasso}} = \sum_{i=1}^n (y_i - \hat{f}(x_i))^2 + \lambda \sum_{j=1}^p |\beta_j|$$

- iv) **Elastic Net:** combinación de Ridge y Lasso.

- 2) **Wrapper methods:**
  - a) Entrenar modelos con **distintas combinaciones de variables** y **evaluar performance**.

- b) Empezar sin variables e ir agregándolas (**Forward Selection**) o empezar con todas las variables e ir sacándolas (**Backward Selection**).
  - c) Se entrenan, siendo  $p$  = número de variables,  $1 + \frac{p(p+1)}{2}$  modelos en la versión greedy del algoritmo.
  - d) Existe una alternativa a Backward Selection que entrena  $p$  modelos aplicando importancia a cada variable. Saca las menos importantes.
- 3) **Filter methods:**
- a) Antes de entrenar HACER la selección.
  - b) Usa tests estadísticos:
    - i) Correlaciones
    - ii)  $\chi^2$
    - iii) ANOVA

Entonces:

- **Filter** → ANTES del entrenamiento
- **Embedded** → DURANTE el entrenamiento
- **Wrapper** → con varios entrenamientos

#### Data leakage:

- 1) **Definición** → información del validation/test set que se FILTRA al entrenamiento. Es como hacer trampa sin querer.
- 2) **Consecuencia** → performance inflada (sobreestimada). Invalida las métricas.
- 3) **¿Cuándo sucede?**
  - a) Selección de atributos usando información de validation/test set (debe hacerse SOLO con training set).
  - b) Escalar usando TODO el dataset (debe hacerse SOLO con training set).
  - c) Oversampling ANTES de separar (debe hacerse DESPUÉS).

---

## Clase VII: Ensamble de Modelos I: Bagging/Random Forest

Habíamos dicho que se entrena un modelo sobre  $X_{tr}$  e  $y_{tr}$ . Conseguimos una estimación de la  $f$  verdadera, llamada  $\hat{f}$ . Probaremos  $\hat{f}$  sobre  $X_{ts}$  e  $y_{ts}$ .

→ El **test error** es el error  $L$  esperado en  $X_{ts}$  dado un training set  $T$ .

$$\rightarrow Err_T = E\left[L\left(Y, \hat{f}(X_{ts})\right) \mid T\right]$$

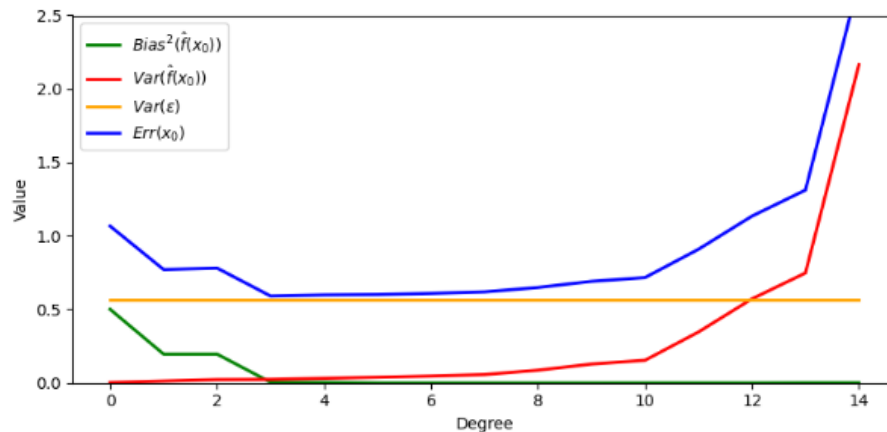
→ **Expected test error:** error **esperado** para una observación nueva (de test) que incluye el proceso generador de datos aleatorio.

$$\rightarrow Err = E\left[L\left(Y, \hat{f}(X_{ts})\right)\right] = E\left[Err_T\right]$$

El error de un modelo se puede descomponer en **sesgo, varianza y ruido irreducible**.

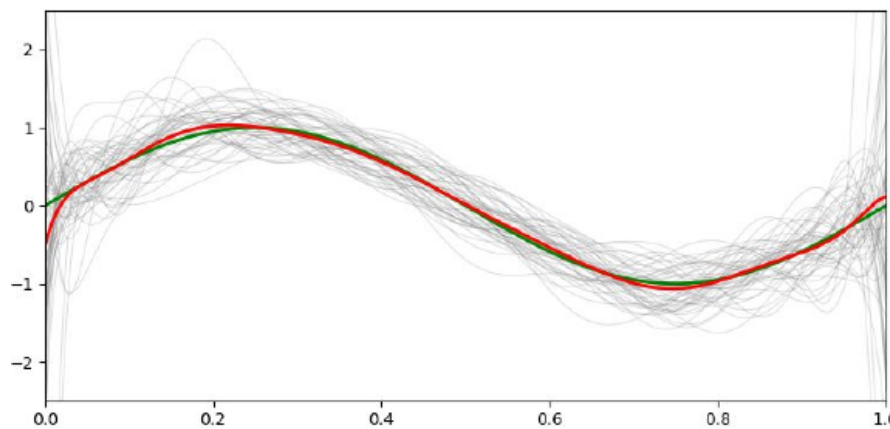
Si estamos en regresión y medimos el error de una observación cualquiera mediante SSE:

$$\begin{aligned}
 Err(x_0) &= E\left[(Y - \hat{f}(x_0))^2 \mid X = x_0\right] \\
 \Rightarrow Err(x_0) &= \sigma_\epsilon^2 + \left[E\hat{f}(x_0) - f(x_0)\right]^2 + E\left[\hat{f}(x_0) - E\hat{f}(x_0)\right]^2 \\
 \Rightarrow Err(x_0) &= \sigma_\epsilon^2 + Sesgo^2(\hat{f}(x_0)) + Var(\hat{f}(x_0)) \\
 \therefore Err(x_0) &= \text{Error Irreducible} + Sesgo^2 + Varianza
 \end{aligned}$$



En un modelo **poco flexible** → alto sesgo, baja varianza → underfitting  
→ Regresión lineal.

En un modelo **muy flexible** → bajo sesgo, alta varianza → overfitting  
→ Regresión polinómica de grado 12.



$f$  verdadera: línea verde

Sesgo: línea roja → bajo sesgo

Varianza: líneas grises → alta varianza

Entonces, el **tradeoff sesgo-varianza** se da si **subimos la flexibilidad** de un modelo (baja el sesgo y aumenta la varianza) o **si la disminuimos** (aumenta el sesgo y baja la varianza).

### Bagging:

- 1) Idea: **reducir la varianza** del modelo (generalmente **regresión**) **promediando muchos modelos**. Queremos acercar la línea roja a la verde lo más posible.
  - a) Generar muchos training set con **bootstrap**. Una muestra con **reposición** del dataset original. **Inventamos nuevas observaciones** con distinto ruido.
    - i) Cada dataset bootstrapeado contiene  $\frac{2}{3}$  del original.  $\frac{1}{3}$  queda out-of-bag.
  - b) Entrenar un modelo **muy flexible** en cada conjunto.

- c) **Promediar** las predicciones → conseguimos el modelo final.
  - i) Bajo sesgo.
  - ii) Baja varianza.
- 2) Ventajas:
  - a) Hacer el promedio nos da una menor varianza.
  - b) El error de test se puede hacer con el out-of-bag error. **No necesitamos validation set**. Probamos en el tercio que quedó afuera del bootstrapping.
- 3) Detalles:
  - a) Entre más modelos  $B$  promediamos, mejor. No genera overfitting.
  - b) **Bagging** es un método de ensamble → combina muchos modelos base para mejorar la performance pero pierde interpretabilidad.
- 4) Problema: los modelos entrenados están **muy correlacionados**, por lo que el promedio no va a reducir tanto la varianza.

#### Random Forest:

- 1) Idea: **no deberían estar correlacionados**. En cambio, usar árboles.
  - a) En cada split del árbol, se elige un subconjunto random de variables.
  - b) Los árboles son distintos en su estructura.
  - c) Se consigue menor correlación y menor varianza.
- 2) Detalles:
  - a) Se suelen usar  $\sqrt{\# \text{ variables}}$  elegidas random por split.
  - b) Uso de hiperparámetros:
    - i) Cantidad de variables por split
    - ii) Profundidad de los árboles
    - iii) Cantidad de árboles
  - c) Se puede usar out-of-bag error y calcular la importancia de las variables.
- 3) Resultados:
  - a) Cada **árbol individual tiene más varianza**.
  - b) El **ensamble** (modelo final) tiene **menos varianza que en bagging**.
  - c) Entrenamiento más rápido.

---

## Clase VIII: Ensamble de Modelos II: Boosting/XGBoost

#### Boosting básico:

- 1) Idea:
  - a) Entrenar un árbol.
  - b) Notar **en qué observaciones predijo mal**.
  - c) Entrenar **otro árbol enfocado en las observaciones que predijo mal** el anterior.
  - d) **Predicción final**: alguna combinación de las predicciones de los árboles.
- 2) Detalles:
  - a) Cada árbol **corrige al anterior**.
  - b) Se trabaja con **árboles de baja profundidad**.
  - c) ¿Cuánto aporta cada nuevo árbol? → Uso de **learning rate**  $\lambda$  → aprendizaje **lento**, pero más estable.
  - d) Cada árbol **depende de todos los anteriores**.

| Inicialización |           |     | Iteración 1 |                     |           |     | Iteración 2 |                     |           |      |
|----------------|-----------|-----|-------------|---------------------|-----------|-----|-------------|---------------------|-----------|------|
| y              | $\hat{f}$ | r   | $\hat{f}^1$ | $\lambda \hat{f}^1$ | $\hat{f}$ | r   | $\hat{f}^2$ | $\lambda \hat{f}^2$ | $\hat{f}$ | r    |
| 10             | 0         | 10  | 8           | 0.8                 | 0.8       | 9.2 | 8.7         | 0.87                | 1.67      | 8.33 |
| 3              | 0         | 3   | 1           | 0.1                 | 0.1       | 2.9 | 2.5         | 0.25                | 0.35      | 2.65 |
| 2              | 0         | 2   | 4           | 0.4                 | 0.4       | 1.6 | 1.2         | 0.12                | 0.52      | 1.48 |
| ...            | ...       | ... | ...         | ...                 | ...       | ... | ...         | ...                 | ...       | ...  |
| 1              | 0         | 1   | 2           | 0.2                 | 0.2       | 0.8 | 1.2         | 0.12                | 0.32      | 0.68 |
| 5              | 0         | 0   | 6           | 0.6                 | 0.6       | 4.4 | 5.1         | 0.51                | 1.11      | 3.89 |

Se ajusta el  $\lambda$  para que el residuo  $r$ , definido como la diferencia entre el valor real y la predicción  $y - \hat{f}$ , sea cada vez menor.

#### Inicialización:

- 1)  $y$ : valores reales de la variable de respuesta.
- 2)  $\hat{f}$ : predicciones actuales del modelo. Inicialmente todas cero.
- 3)  $r$ : residuo  $\rightarrow y - \hat{f} = 10 - 0 = 10$ .

#### Iteración 1:

- 1)  $\hat{f}^1$ : se entrena el primer árbol para predecir los residuos  $r$  de la inicialización.
- 2)  $\lambda \hat{f}^1$  con  $\lambda = 0,1 \rightarrow \lambda \hat{f}^1 = 0,1 \times 8 = 0,8$ .
- 3)  $\hat{f}$ : nueva predicción. Predicción anterior +  $\lambda \hat{f}^1 \rightarrow \hat{f} := \hat{f} + \lambda \hat{f}^1 := 0 + 0,8 = 0,8$ .
- 4)  $r$ : nuevo residuo  $\rightarrow y - \hat{f} = 10 - 0,8 = 9,2$

#### Iteración 2:

- 1)  $\hat{f}^2$ : se entrena el segundo árbol para predecir los residuos  $r$  de la iteración 1.
- 2)  $\lambda \hat{f}^2$  con  $\lambda = 0,1 \rightarrow \lambda \hat{f}^2 = 0,1 \times 8,7 = 0,87$ .
- 3)  $\hat{f}$ : nueva predicción. Predicción anterior +  $\lambda \hat{f}^2 \rightarrow \hat{f} := \hat{f} + \lambda \hat{f}^2 := 0,8 + 0,87 = 1,67$ .
- 4)  $r$ : nuevo residuo  $\rightarrow y - \hat{f} = 10 - 1,67 = 8,33$ .

Y así repetir sucesivamente...

### **XGBoost (Extreme Gradient Boosting):**

- 1) Acerca de:
  - a) Modelo y librería de boosting usado en **regresión y clasificación**.
  - b) Función objetivo:
    - $\rightarrow Obj(\theta; X, y) = L(\theta; X, y) + \Omega(\theta)$
    - $\rightarrow$  Siendo  $L$  el **error de entrenamiento** (SSE o cross-entropy).
    - $\rightarrow$  Siendo  $\Omega$  un parámetro de **regularización**  $\rightarrow$  **penaliza complejidad**.
- 2) Ensamble de árboles:
  - a) La predicción final es la suma de las predicciones de  $K$  árboles.
  - b) Cada árbol le asigna un valor a cada hoja.
  - c) Se hace un entrenamiento aditivo y secuencial.
 
$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i) \equiv \sum_{k=1}^t f_k(x_i)$$
  - d) Optimización usando series de Taylor.
    - i) Usa gradientes  $g_i$  de la pérdida
    - ii) Usa segundas derivadas  $h_i$  de la pérdida
  - e) Regularización dependiendo del número de hojas y sus valores.

$$\rightarrow \Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_j w_j^2$$

- f) Criterio para decidir splits  $\rightarrow$  **ganancia**.
- 3) Missings:
- Prueba si asignar los NA a la izquierda o a la derecha del split le da más ganancia, y elige esa dirección.
- 4) Algoritmo:
- Calcular, para cada observación,  $g_i$  y  $h_i$ .
  - Construir un árbol greedy maximizando la ganancia.
  - Calcular valores óptimos  $w_j$  de las hojas.
  - Agregar el árbol al ensamble (puede ser con un factor de aprendizaje  $\eta$ ).
  - Repetir hasta completar un número de iteraciones.
- 5) Hiperparámetros:
- nrounds**: número de árboles.
  - max\_depth**: profundidad máxima de cada árbol.
  - eta**: learning rate.
  - gamma**: reducción mínima de error para hacer un split.
  - colsample\_bytree**: fracción de variables usadas en cada árbol.
  - min\_child\_weight**: mínimo de observaciones en un nodo hijo.
  - subsample**: fracción de observaciones usadas en cada árbol.

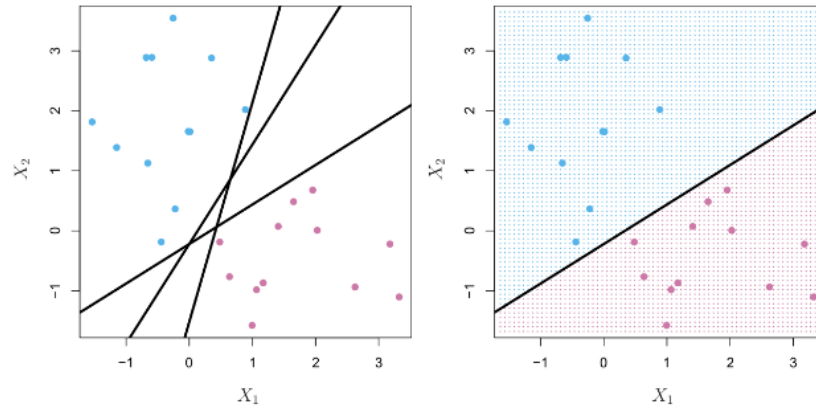
## Clase IX: Regresión Logística/SVMs

### Regresión Logística:

- 1) Función sigmoidea:
- $$\rightarrow P(y_i = 1 | x_{i1}, x_{i2}, \dots, x_{im}) = \left[ 1 + e^{-(\beta_0 + \beta_1 x_{i1} + \dots + \beta_m x_{im})} \right]^{-1}$$
- $$\rightarrow \hat{P}(y_i = 1 | x_{i1}, x_{i2}, \dots, x_{im}) = \left[ 1 + e^{-(\hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \dots + \hat{\beta}_m x_{im})} \right]^{-1}$$
- 2) **Cross-entropy**: mide qué tan buena es la predicción de las probabilidades.
- Queremos encontrar los betas que minimicen el cross-entropy.
  - Minimizar cross-entropy ES maximizar la verosimilitud.
- 3) **Frontera de decisión**: el modelo separa el espacio de atributos con una línea/hiperplano.
- 4) **Estimación de parámetros**: no hay fórmula cerrada, se usan métodos numéricos  $\rightarrow$  descenso gradiente.
- 5) **Regularización**:
- Se implementa Cross-Entropy + Ridge.
  - Penaliza la complejidad del modelo.
- $$\rightarrow CE_{Reg} = CE + \frac{\lambda}{2} \sum_j \theta_j^2$$
- Un  $\lambda$  grande  $\rightarrow$  coeficientes  $\beta$  pequeños.
  - No se penaliza el término de sesgo.

### **SVMs (Support Vector Machines):**

- 1) Existe un hiperplano en dimensión  $p$  (número de variables).  
 $\rightarrow \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p = 0$
- 2) Objetivo: queremos un hiperplano (línea negra) que separe las clases  $y_i = 1$  de  $y_i = -1$ .
  - a) De un lado debe estar  $y_i = 1$ .
  - b) Del otro lado debe estar  $y_i = -1$ .
  - c) Si los datos se pueden separar por una única línea, existen infinitos hiperplanos posibles que los separen.



- 3) El **margen** es la distancia del punto más cercano a la frontera.
  - a) El mejor hiperplano será el que tenga mayor margen. Separa “más tranquilo” a los datos. El hiperplano que maximice un margen se llama **Maximal Margin Classifier**.
- 4) Los puntos SOBRE el hiperplano forman el **support vector**.
- 5) Pasos a seguir:
  - a) Buscamos el margen  $M$  más grande.  
 $\rightarrow \max_{\{\beta_0, \beta_1, \dots, \beta_p, M\}} M$   
 $\rightarrow$  Sin restricciones se iría a infinito.
  - b) Restricciones.
    - i) La suma de todos los coeficientes al cuadrado da 1.  
 $\rightarrow \text{sujeto a } \sum_{j=1}^p \beta_j^2 = 1$
    - ii) La distancia de cada punto al hiperplano debe ser mayor o igual al margen. Si no es linealmente separable el dataset, las restricciones no tienen solución.  
 $\rightarrow y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M, \forall i = 1, \dots, n$
- 6) Casos No Linealmente Separables:  
 Usamos **Support Vector Classifier (Soft Margin)**.
  - a) **Permite errores**.
  - b) Variables slack  $\epsilon_i$ :
    - i)  $\epsilon_i = 0 \rightarrow$  bien clasificado
    - ii)  $0 < \epsilon_i < 1 \rightarrow$  mal clasificado respecto al margen
    - iii)  $\epsilon_i > 1 \rightarrow$  mal clasificado respecto al hiperplano



- c) Parámetro C: controla la **tolerancia** a errores.
  - i) C **grande** → tolera **más** errores
  - ii) C **chico** → tolera **menos** errores
- 7) **Kernels y no linealidades**:
  - a) Kernel: función que toma dos observaciones y devuelve el valor del producto interno de ellas en un espacio expandido.
  - b) Kernel trick: reemplazar el producto interno estándar por una función de similitud  $K(x_i, x_j)$ .
    - i) Permite trabajar en espacios de atributos de mayor dimensión sin calcular explícitamente la expansión
  - c) Tipos de kernels:
    - i) **Polinómico** → genera fronteras no lineales
    - ii) **Radial Basis Function (RBF)** → la medida  $K$  decrece con la distancia euclídea

---

## Clase X: Bayes Ingenuo y KNN

### Clases de modelos:

- 1) Paramétricos:
  - a) Las variables en el dataset  $X$  se relacionan de manera fija y predefinida con la respuesta.
  - b) Imponemos nosotros una función (modelo) que trate de explicar la relación.
  - c) Hay un número finito de parámetros.
    - i) Regresión lineal → se mantiene siempre la misma forma  
→  $y \approx \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$
    - ii) Regresión logística
- 2) No paramétricos:
  - a) No existe una forma que diga cómo se relacionan las variables con la respuesta.
  - b) No imponemos una función → el modelo se adapta a los datos.
  - c) La complejidad crece con los datos.
  - d) No hay un número fijo de parámetros.
    - i) KNN
    - ii) SVM con RBF
- 3) Discriminativos:
  - a) Modelan la probabilidad condicional.
    - i) Árboles de decisión
    - ii) Regresión logística
- 4) Generativos:
  - a) Approach menos directo que un modelo discriminativo.
  - b) Estiman  $P(X | y)$  y  $P(y)$ .
  - c) Usan el Teorema de Bayes.
  - d) Pueden simular observaciones.

### Bayes Ingenuo:

- 1) Acerca de:

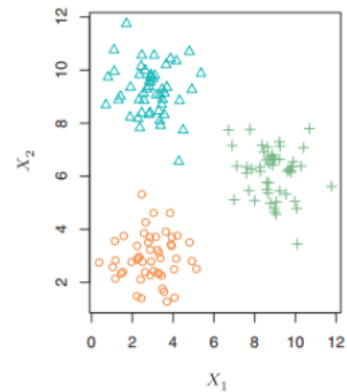
- a) Probabilidad **prior**  $\rightarrow P(C_k)$ .
  - b) Algoritmo de clasificación generativo.  

$$\rightarrow P(C_k | x_i) = \frac{P(x_i | C_k) \times P(C_k)}{P(x_i)}$$
  - c) **Supuesto ingenuo**  $\rightarrow$  independencia condicional entre variables **categorías** dada una clase.  

$$\rightarrow P(C_k | X_i) \text{ es proporcional a } P(C_k) \prod_j P(v_j = x_{ij} | C_k)$$
  - d) **Problema**  $\rightarrow$  si una probabilidad condicional es cero, toda la productoria da cero.
    - i) **Solución**  $\rightarrow$  **suavizado aditivo**. Agrega una observación fantasma para evitar ceros
- 2) Adaptaciones para variables continuas:
- a) Discretización  $\rightarrow$  como en ingeniería de atributos.
  - b) Funciones de densidad  $\rightarrow$  paramétricas o no paramétricas.
- 3) Ventajas:
- a) Simple y rápido.
  - b) Funciona bien con pocos datos.
  - c) Ocupa poco espacio.
  - d) Los atributos irrelevantes no molestan.
- 4) Desventajas:
- a) Supuesto de independencia  $\rightarrow$  poco realista.
  - b) No captura relaciones complejas entre las variables.

### K-Nearest-Neighbors (KNN):

- 1) Características: modelo **no paramétrico** y **discriminativo**. Se puede usar para **clasificación** y para **regresión**.
- 2) En clasificación:
  - a) Definir una medida de distancia.
    - i) Euclídea
    - ii) Manhattan
    - iii) Hamming
    - iv) Mahalanobis
  - b) Para un nuevo punto  $x_0$ , buscar sus  $K$  vecinos más cercanos.
  - c) Predecir la clase según la mayoría de esos vecinos.
- 3) Acerca del  $K$ :
  - a)  $K$  chico  $\rightarrow$  frontera flexible (riesgo de overfitting).
  - b)  $K$  grande  $\rightarrow$  frontera rígida (se acerca a predecir los priors).
- 4) Problema: **Curse of Dimensionality**.  
 Si tenemos muchas variables es difícil encontrar vecinos cercanos que nos sirvan.
- 5) Consideraciones:
  - a) Estandarizar variables.
  - b) Para variables categóricas  $\rightarrow$  usar OHE.
- 6) En regresión:
  - a) No se usa más el criterio de mayoría.
  - b) Se usa el **promedio de la respuesta y de los vecinos**.



- 7) Ventajas:
- a) Simple.
  - b) Flexible con las métricas.
  - c) Intuitivo para clasificación multiclase.
- 8) Desventajas:
- a) **Costoso en predicción** → compara el dato nuevo con todos los anteriores.
  - b) Almacenar un KNN es muy **costoso en cuanto al peso**.

**Bayes Ingenuo vs. KNN:**

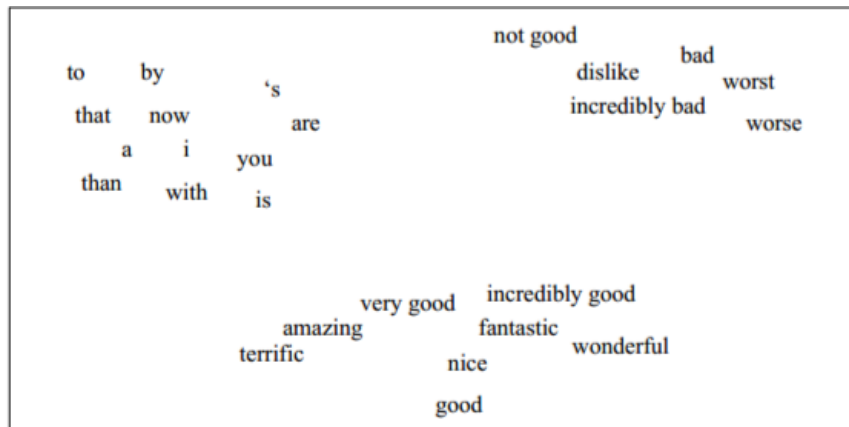
| Bayes Ingenuo                                                                                                         | KNN                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Eager learning</b> → aprende cuando entrena.<br>Guarda un resumen en forma de modelo.<br><b>Predicción rápida.</b> | <b>Lazy learning</b> → no aprende cuando entrena. Trabaja sólo cuando predice.<br><b>Predicción costosa.</b> |

## Clase XI: Word2vec

**Vector semantics:** palabra → vector en un espacio multidimensional. Captura algo de su significado.

**Word embeddings** → representar palabras como vectores. Implementación de vector semantics. Vectores resultantes del entrenamiento de un modelo. Word2vec es el modelo conocido.

Son parecidas → estarán cerca.



Fundamento: las palabras que aparecen en **contextos similares** tienden a tener **significados similares**. Hipótesis distribucional.

→ “Veterinaria” aparece en contextos de “gato” y “perro”.

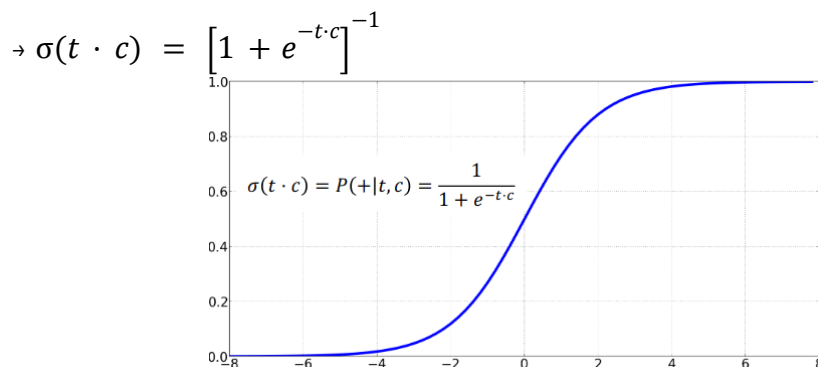
→ “Veterinaria” no aparece en contextos de “álgebra”.

**Construcción de embeddings:**

- 1) Se define el **contexto** de una palabra como las  $n$  palabras anteriores y posteriores en la oración (corpus).
- 2) Queremos entrenar **un modelo que aprenda a decirnos si dos palabras tienden a aparecer juntas**.
- 3) No nos interesa la predicción del modelo, sino los **vectores** que aprende.

### Se usa el siguiente modelo de Word2vec:

- 1) Es parecido a una regresión logística.
- 2) Para un par  $(t, c)$ :
  - a)  $t = \text{target word}$ ,  $c = \text{context word}$ ,  $+$  = *positivo*.
  - b)  $P(+ | t, c) \rightarrow$  **alto** si **comparten** contexto.
    - i)  $t = \text{"Veterinaria"}$ ,  $c = \text{"Perro"}$  cercano a uno
  - c)  $P(+ | t, c) \rightarrow$  **bajo** si **no comparten** contexto.
    - i)  $t = \text{"Veterinaria"}$ ,  $c = \text{"Álgebra"}$  cercano a cero
- 3) ¿Similitud?  $\rightarrow$  producto interno  $t \cdot c$ .
- 4) Uso de función sigmoidea para conseguir la probabilidad:



### Datos de entrenamiento:

- 1) Casos positivos: palabras que Sí aparecen en la ventana  $n$ .
  - a) Ventana  $n = 1$
  - b)  $t = \text{"gato"}$
  - c) Contexto = "El gato duerme en la medianera"  $\rightarrow c = \{\text{"El"}, \text{"duerme"}\}$ 
    - i)  $t = \text{"gato"}, c_{pos} = \text{"El"} \rightarrow$  positivo
    - ii)  $t = \text{"gato"}, c_{pos} = \text{"duerme"} \rightarrow$  positivo
- 2) Casos negativos: para cada positivo se muestran  $k$  palabras "ruido" que no son target. **Se eligen al azar.**
  - a) Ventana  $n = 1$ .
  - b)  $t = \text{"gato"}$ .
  - c) Contexto = "El gato duerme en la medianera"  $\rightarrow c = \{\text{"El"}, \text{"duerme"}\}$ .
    - i)  $t = \text{"gato"}, c_{neg} = \text{"vándalo"} \rightarrow$  negativo
    - ii)  $t = \text{"gato"}, c_{neg} = \text{"España"} \rightarrow$  negativo

### Objetivo:

- 1) **Encontrar un  $\theta$  que maximice** -sujeto a  $L(\theta)$ - la probabilidad de **positivos** y **minimice** la de **negativos**.
  - a)  $\theta =$  valores de  $T$  y  $C$ .
- 2) Un aprendizaje con descenso de gradiente estocástico sobre matrices  $T \in \mathbb{R}^{V \times d}$  y  $C \in \mathbb{R}^{d \times V}$ .
  - a) Cada fila  $t_i$  de  $T$  es el vector del vocabulario **cuando la palabra está como target**.
  - b) Cada columna  $c_j$  de  $C$  es el vector de la palabra **cuando está como contexto**.
- 3) En definitiva, importan los embeddings aprendidos  $T$  y  $C$ .

### Optimización:

- 1) Captura analogías:
  - a) “Rey” - “Hombre” + “Mujer”  $\approx$  “Reina”.
  - b) “París” - “Francia” + “Argentina”  $\approx$  “CABA”.
- 2) Importa el tamaño de la ventana  $n$ :
  - a)  $n$  chico  $\rightarrow$  similitud semántica.
  - b)  $n$  grande  $\rightarrow$  relación temática.

---

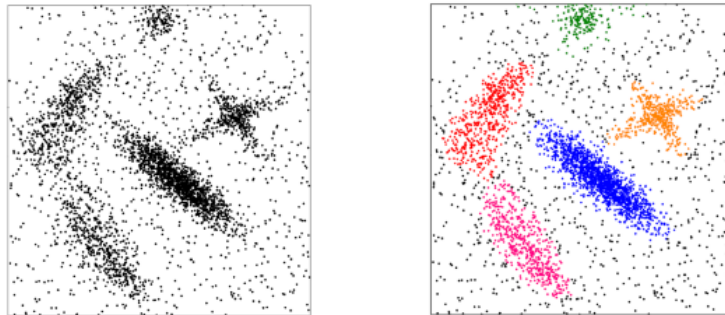
## Clase XII: Clustering

$\rightarrow$  Buscamos subgrupos (**clusters**) dentro de un dataset.

**Dentro** de un cluster  $\rightarrow$  observaciones similares.

**Entre** clusters  $\rightarrow$  observaciones serán distintas.

¿Qué quiere decir similar?  $\rightarrow$  depende de dónde apliquemos el modelo.

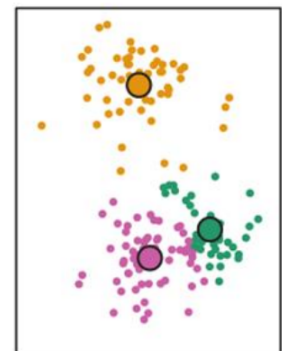


Familias de algoritmos de clustering:

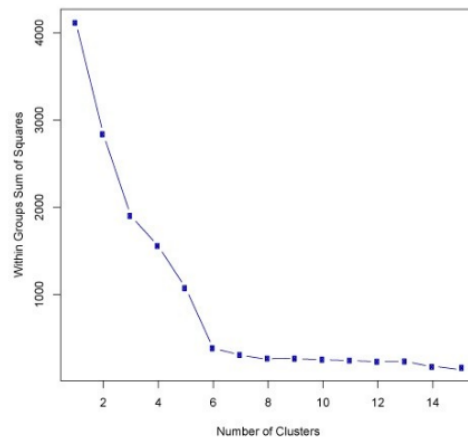
- 1) Partitioning: divide en **clusters excluyentes**. **K-Means**.
- 2) Jerárquico: **estructura jerárquica** de clusters. **AGNES**.
- 3) Density-based: agrupa observaciones en áreas de alta densidad.
- 4) Grid-based: divide el espacio en grillas para crear clusters.
- 5) Model-based: supone un modelo estadístico para crear clusters..

**K-Means Clustering:**

- 1) Algoritmo tipo partitioning.
- 2) Se crean  $K$  clusters  $\rightarrow$  valor definido antes.
- 3) No existen intersecciones entre clusters  $\rightarrow$  son excluyentes.
- 4) Queremos **minimizar la variabilidad**  $W(C_k)$  de las observaciones dentro de un cluster.
  - a) Medición  $\rightarrow$  distancia euclídea al cuadrado.
  - b) Se define un **centroide** por cluster. Círculo marcado en los clusters.
- 5) Algoritmo:
  - a) Elegir  $K$  y los centroides iniciales.
  - b) Asignar cada observación al centroide más cercano.
  - c) Recalcular los centroides.
  - d) Repetir hasta converger.
- 6) Consideraciones:
  - a) El resultado depende de la inicialización  $\rightarrow$  aleatorio.



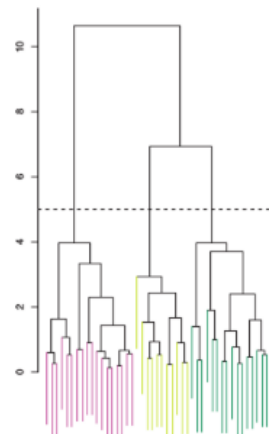
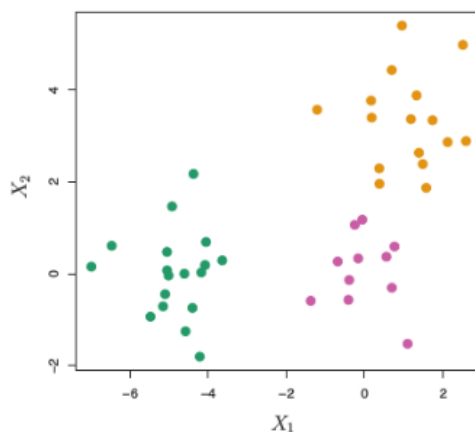
- b) Conviene correr el algoritmo varias veces → elegimos la mejor solución.
- c) ¿Qué  $K$  elegimos?
  - i) Corremos el algoritmo muchas veces con  $K$  creciendo cada vez



- ii) Donde se forme **un codo**, será el óptimo de  $K \rightarrow K = 6$
- d) Es un algoritmo sensible al escalado de variables.

### **Clustering Jerárquico (AGNES):**

- 1) Algoritmo aglomerativo → junta clusters más similares en uno solo.
- 2) No es necesario que fijemos  $K$  de antemano.
- 3) Conseguiremos un dendrograma.
  - a) Hoja → observación.
  - b) Se unen grupos de observaciones según su similitud.
  - c) Entre más abajo se unen los grupos → más parecidos -ceranos- son los elementos.
- 4) La definición de distancia entre clusters cambia el resultado.
- 5) Tipos de linkage:
  - a) **Single linkage** → **mínima** distancia entre un par de puntos.
  - b) **Complete linkage** → **máxima** distancia entre un par de puntos.
  - c) **Average linkage** → **promedio de todas las distancias** entre pares de puntos de los dos clusters.



### **Ejemplo:**

Supongamos 4 observaciones.

- **A(1); B(2); C(8); D(10)**

- 1) Paso inicial: cada punto es un cluster.
  - {A}, {B}, {C}, {D}

- 2) Calcular distancias entre clusters.
  - $\text{Dist}(A, B) = |1-2| = 1$
  - $\text{Dist}(A, C) = |1-8| = 7$
  - $\text{Dist}(A, D) = |1-10| = 9$
  - $\text{Dist}(B, C) = |2-8| = 6$
  - $\text{Dist}(B, D) = |2-10| = 8$
  - $\text{Dist}(C, D) = |8-10| = 2$
- 3) Unir los más cercanos.
  - El mínimo es 1  $\rightarrow$  unimos A y B
  - Tenemos los clusters  $\{A, B\}, \{C\}, \{D\}$
- 4) Calcular distancias (en este caso con single linkage).
  - $\text{Dist}(\{A, B\}, C) = \min(7, 6) = 6$ 
    - Optamos entre  $\min(\text{Dist}(A, C); \text{Dist}(B, C))$
  - $\text{Dist}(\{A, B\}, D) = \min(9, 8) = 8$
  - $\text{Dist}(C, D) = 2$
- 5) Unir los más cercanos.
  - El mínimo es 2  $\rightarrow$  Unimos C y D
  - Tenemos los clusters  $\{A, B\}, \{C, D\}$
- 6) Calcular distancias.
  - $\text{Dist}(\{A, B\}, \{C, D\}) = \min(6, 8) = 6$

## Clase XIII: PCA

**Varianza**  $\rightarrow$  mide la **dispersión** de una variable respecto a su media.

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

**Covarianza**  $\rightarrow$  mide la **relación lineal** entre dos variables.

$$\text{cov}(X_j, X_k) = \frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$$

$\rightarrow$  La unidad de medida de las variables afecta el resultado. Por eso usamos el coeficiente de correlación de Pearson, que va desde  $[-1, 1]$ .

$$\rho_{jk} = \frac{\text{cov}(X_j, X_k)}{\sigma_{X_j} \sigma_{X_k}}$$

La **matriz de covarianzas** tiene las **varianzas en la diagonal** y las **covarianzas fuera de la diagonal**. A la suma de todas las varianzas se la llama varianza total.

La **matriz de correlaciones** tiene **unos en la diagonal** y los **coeficientes de Pearson fuera de la diagonal**.

### Análisis de Componentes Principales (PCA):

- 1) Podemos tener muchas variables en un dataset  $\rightarrow$  se complica para graficar porque tenemos muchas dimensiones.
- 2) Buscamos **reducir las dimensiones** manteniendo la **mayor varianza posible**.
- 3) Es un método no supervisado.
- 4) Nuevos ejes  $\rightarrow$  componentes principales  $\rightarrow$  combinaciones lineales de las variables originales.

5) Componentes principales:

- a) Primer componente  $\rightarrow Z_1$ .
  - i) Combinación lineal normalizada de variables
  - ii) Maximiza la varianza
  - iii) ¿Cómo hallamos los coeficientes?  $\rightarrow$  Problema de optimización
- b) Segundo componente  $\rightarrow Z_2$ .
  - i) Maximiza la varianza RESTANTE
  - ii) Ortogonal (no correlacionado) a  $Z_1$
- c) Podemos tener **tantos componentes como variables** en el dataset.

6) ¿Qué tiene un componente principal?

$$\rightarrow Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \dots + \phi_{p1}X_p$$

- a)  $\phi_{jm}$  son los pesos que indican cuánto aporta la variable original  $X_j$  al  $m$ -ésimo componente principal  $Z_m$ .
- b) Cada componente es una combinación lineal con pesos  $\phi$  de las variables.
- c) Restricción: vector unitario.

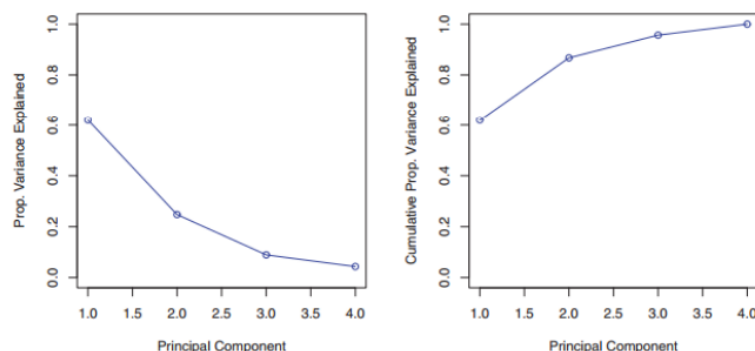
$$\rightarrow \sum_{j=1}^p \phi_{j1}^2 = 1$$

Interpretando PCA:

- 1) En el plano de componentes:
  - a) La posición de las observaciones marca la similitud entre ellas.
  - b) La dirección de las flechas marca la correlación entre las variables.
- 2) PCA busca superficies lineales de **baja dimensión que se aproximen a los datos originales**.
- 3) La **escala importa**  $\rightarrow$  estandarizar antes de PCA.

Varianza explicada:

- 1) **Varianza total original**  $\rightarrow$  suma de las varianzas de las variables.
- 2) **Varianza explicada por el componente  $m$**   $\rightarrow$  **cuánta información reúne** ese eje.
- 3) **Proporción de la varianza explicada**  $\rightarrow Var(Z_m) \div Var_{Total}$
- 4) Uso de **scree plot** para ver la varianza explicada por cada componente.



¿Cuántos componentes precisamos?  $\rightarrow$  No hay un criterio único.

Podríamos buscar el codo (de nuevo) en el scree plot, o bien tener componentes que -juntos- expliquen mínimo 80% de la varianza.